

graph-rag: CPG-Based Vulnerability Retrieval

Technical Progress Summary for Stakeholders

Master's Thesis — TU Munich — Siemens AI Engineering

Motivation & Problem Statement

Identifying whether a new code snippet contains a known vulnerability pattern requires searching across thousands of historical CVEs efficiently. Classical text search misses structural code patterns; pure LLM-based approaches lack precise structural grounding. This project builds a **retrieval-augmented generation (RAG) system indexed on Code Property Graphs (CPGs)**, enabling similarity search at the level of program structure rather than surface text.

Data Pipeline

Currently Implemented

- CPG Export via Joern.** C/C++ source functions are parsed with `joern-parse` and exported as GraphML using `joern-export --repr all`. Each function produces a directory of `export.xml` files covering AST, CFG, CDG, and PDG edges.
- Graph Loading & Merging.** All `export.xml` files under a function directory are merged into a single `MultiDiGraph` (`NetworkX`). Phantom nodes (auto-created by edge references to missing nodes) are identified by tracking declared vs. referenced node IDs and removed. `COMMENT` and `UNKNOWN` nodes are filtered.
- Semantic Graph Diff.** Vulnerable (*before*) and patched (*after*) CPGs are aligned using **fingerprint-based node matching** — nodes are matched by (`labelV`, `normalised CODE`) rather than Joern-assigned IDs, which are non-deterministic across runs. Changed nodes are classified as **removed**, **added**, **mutated** (same structure, different code), or **rewired** (edge topology changed). Each node in the vulnerability-relevant subgraph (G_{vuln}) receives a `diff_weight` $\in [0, 1]$ encoding its change significance.
- Dataset Loaders.** Three datasets are supported via a common `BaseDataset` interface that yields `FunctionPair` objects:
 - **BigVul** — CSV with `func.before/func.after` columns.
 - **CVEFixes** — SQLite database with `method.change` table.
 - **AutoPatch** — folder-per-CVE with original source files and LLM-generated augmented variants (augmented, re-implemented by GPT-4o, DeepSeek, LLaMA, o3-mini).

All loaders share a `export_jobs()` method that feeds the Joern export step, and a `stream()` method for index building.

- Multiprocessing Joern Export.** `main.py --mode export` parallelises Joern invocations across functions using a `multiprocessing.Pool`. Skip-if-exists logic makes re-runs safe after partial failures.
- Embedding & Indexing.** Extracted graphs are embedded and stored in a FAISS index with aligned JSON metadata (CVE ID, CWE, function name, dataset, variant).
- Experiment Framework.** A runner evaluates all (`embedder`, `backend`, `graph_variant`) combinations and computes self-retrieval, leave-one-out, CWE-group recall, query latency, and embedding space statistics. Results are exported to JSON and visualised in a standalone HTML dashboard.

Next Steps

- PDG density validation.** Confirm that `REACHING.DEF` and CDG edges are present in exports (`--repr all` flag). Currently $> 50\%$ of pairs fall back to CFG-only slicing.

2. **Ground-truth query labels.** Construct a held-out query set with known CVE targets for honest precision/recall evaluation.
3. **Temporal generalisation split.** Train on CVEs before year Y , query with year $Y+1$ CVEs to measure generalisation to unseen patterns.
4. **Agent integration.** Wire the `Retriever` into ADK agents for end-to-end patch suggestion.
5. **Scaling.** Extend to full BigVul ($\sim 3,500$ CVEs) and CVEFixes ($\sim 10,000$ method pairs).

RAG Index Backends

1. **FAISS Flat (IndexFlatIP).** Exact inner-product search over L2-normalised vectors (equivalent to cosine similarity). No approximation; guarantees correct top- k . Scales to $\sim 100k$ vectors on CPU without issue. Used as the correctness baseline in all experiments.
2. **HNSW (Hierarchical Navigable Small World).** Approximate nearest-neighbour index via a layered proximity graph. Sub-linear query time $O(\log n)$ vs. linear for flat search. Configurable via `M` (graph connectivity) and `ef_construction/ef_search` (recall/speed trade-off). Preferred backend for production at $>10k$ vectors. Both backends share an identical interface (`add`, `save`, `load`) and are interchangeable in config.

Graph Extraction Strategies

1. **ID-based diff (baseline, deprecated).** Original approach: set difference of Joern node IDs between *before* and *after* graphs. Produces $G_{\text{vuln}} \approx G_{\text{before}}$ because Joern assigns different IDs to semantically identical nodes across runs.
2. **Semantic diff with n -hop expansion.** Fingerprint-matched diff with configurable neighbourhood expansion ($n \in \{0, 1, 2, 3\}$ hops). $n = 0$ returns only directly changed nodes; $n = 3$ adds structural context. Experiments show $n = 1$ or $n = 2$ best balances signal density and context.
3. **CFG-only subgraph.** Retains only control-flow edges and their endpoints. Reduces AST noise (AST edges dominate most CPGs) and focuses on execution path structure.
4. **PDG slice (S1).** Follows `REACHING_DEF`, `CDG`, and `REF` edges from changed nodes via bidirectional BFS. Captures data and control dependencies — theoretically the most semantically rich representation for vulnerability detection (Yamaguchi et al. 2014, Ferrante et al. 1987). Falls back to CFG automatically when PDG edge density $< 10\%$.
5. **Change-anchored PDG hop (S2).** Restricts n -hop expansion to semantic edge types only (`CFG` $\cup$ `CDG` $\cup$ `REACHING_DEF`). Eliminates AST-dominated context that inflates G_{vuln} without adding vulnerability signal.
6. **Vulnerability pattern motifs (S3).** Counts occurrences of 8 hand-crafted 3-node motifs around changed nodes (e.g. `CALL` $\rightarrow$ `IDENTIFIER` via `REACHING_DEF` for use-after-free; `CALL` $\rightarrow$ `CONTROLSTRUCTURE` via `CFG` for missing checks). Returns a normalised 8-dimensional histogram — CWE-agnostic and independent of graph topology.

Embedders

All embedders are **unsupervised** (no labelled training data required) and output L2-normalised vectors of configurable dimension (default 128).

1. **NetLSD (Network Laplacian Spectral Descriptor).** Computes the heat trace of the normalised graph Laplacian across a log-spaced timescale range. Permutation-invariant; captures global spectral structure. Sensitive to graph size — collapses on graphs with < 3 nodes. Weighted variant uses `diff_weight` to bias edge weights toward changed nodes.

2. **WL Baseline** (*Weisfeiler-Lehman*). Iterative colour refinement over node types followed by weighted sum pooling and a linear projection. Equivalent in expressive power to graph2vec’s subtree kernel but implemented in PyTorch Geometric with no external dependency. **diff_weight** biases pooling toward high-signal nodes.
3. **GIN** (*Graph Isomorphism Network*). 3-layer GIN with random (frozen) weights. Random projections preserve pairwise distances (Johnson–Lindenstrauss) and provide stronger structural fingerprints than WL on small graphs due to MLP non-linearity. Dual pooling (sum + mean concatenated).
4. **Combined**. Concatenates NetLSD + WL + GIN descriptors ($3 \times 128 = 384$ dimensions) and reduces via PCA to the target dimension. Explained variance reported at fit time. Most robust to individual embedder collapse.
5. **Motif Embedder**. Concatenates: (i) motif histogram (8-dim), (ii) semantic CODE features (ptr/alloc/free/lock/cast counts per node, 12-dim), (iii) node and edge type histograms (15-dim), (iv) NetLSD on PDG slice (16-dim). Total ~ 51 raw features projected to target dimension via Gaussian random projection (no training). Designed to be discriminative when pure topology is not.

Experiments & Early Results

Experiment Grid

All experiments evaluate the full (**embedder**, **backend**, **graph_variant**) grid. Metrics: **Hit@K** (fraction of queries where the correct CVE appears in top- K results), **MRR** (mean reciprocal rank), **CWE-group recall** (fraction of top- K results sharing the query’s CWE type), **effective dimensionality** of the embedding space, and **query latency** (p50/p99).

Graph Strategy Experiments (E1–E8)

- **E1** — diff with $n \in \{0, 1, 2, 3\}$ hops.
- **E2** — subgraph strategies: CFG-only, PDG slice, method-entry BFS, full G_{before} .
- **E3** — CFG topological order as node feature.
- **E4** — semantic CODE token features on G_{vuln} .
- **E5** — pure semantic vector (no topology).
- **E6** — hybrid: PDG slice + semantic features.
- **E7** — PDG slice + motif embedder.
- **E8** — motif histogram on PDG slice.

Results Summary

Preliminary findings on BigVul subset (~ 200 CVEs):

- PDG slice (S1) with GIN+NetLSD achieves **Hit@10 = 0.68**.
- Motif embedder (S3) is most robust to graph collapse; suitable for production at scale.
- HNSW (M=16, ef=100) achieves **<5ms p99 latency** with <2% recall loss vs. FAISS flat.
- CWE-group recall reaches 0.55, indicating structural patterns are indeed recoverable.

Next Milestones

1. Scale to full BigVul and CVEFixes.
2. Integrate with ADK agent framework.
3. Temporal split evaluation.
4. Dashboard visualisation and open-source release.

This is a ****bounded BFS** (breadth-first search) program slice** starting from the seed nodes identified in steps 1–2.

What it does: Starting from every node in **changed** (removed nodes, fix-adjacent nodes, edge-changed nodes), it expands outward up to $SliceDepth = 3$ hops, but only along flow edges: CFG, CDG, REACHING_DEF, PDG, DDG. It follows edges in both directions (successors via **out_edges** and predecessors via **in_edges**).

****Why:**** The **changed** set alone only contains the directly modified nodes. But a vulnerability’s behavior depends on surrounding control/data-flow context — e.g. a use-after-free involves a **free()** call (changed) **and** the subsequent **use()** (unchanged but reachable via REACHING_DEF).

The 3-hop expansion captures that neighborhood while staying bounded.

****Walk-through of one iteration:**** 1. **frontier** starts as all **changed** nodes 2. For each node in **frontier**, collect all neighbors reachable by one flow edge that aren’t already in **slice_nodes** → that’s **next_frontier** 3. Add **next_frontier** to **slice_nodes**, then repeat with **next_frontier** as the new frontier 4. After 3 rounds, **slice_nodes** contains everything within 3 flow-edge hops of any changed node

The result is the vulnerability-relevant ****program slice**** — the subgraph of **G_before** that is transitively connected to the patch through data/control dependencies, bounded to avoid pulling in the entire CPG.

Experiments

Dataset and Evaluation Protocol

All experiments are conducted on the **AutoPatch** dataset, comprising 75 original Linux kernel CVE functions and their LLM-generated re-implementations (5 variants per CVE: augmented, GPT-4o, DeepSeek, DeepSeek-R1, LLaMA, o3-mini), yielding **225 index pairs** after filtering pairs with failed Joern exports.

We use a **stratified train/test split** (seed = 42, test ratio = 0.2) that partitions the augmented variants by CWE class to ensure each class is represented in both sets. All 75 original CVE pairs are retained in the index; **73 held-out augmented pairs** form the query set. At query time, the system must retrieve a same-CVE item from the 225-pair index for each of the 73 queries.

Metrics. We report the following, all computed at $k=5$ unless stated otherwise:

- **hit@k**: fraction of queries whose correct CVE appears in the top- k retrieved results (micro-averaged).
- **MRR**: mean reciprocal rank of the first correct CVE across all queries.
- **CWE Recall**: for each query, the fraction of same-CWE items retrieved in top- k , macro-averaged across CWE classes.
- **Effective dimensionality**: the number of PCA components needed to explain 90% of the embedding variance, measuring how many independent directions the embedder uses.

Embedders. Six unsupervised embedders are evaluated: three *structural* (Combined = NetLSD || WL || GIN concatenated and PCA-projected to 128-d; GIN alone; WL alone), two *semantic* (CodeBERT-seq using the [CLS] token of changed-node code; CodeBERT-pattern adding 34 structural features), and one *hybrid* (RGCN, a relational GCN with edge-type-specific weight matrices). All embedders output L2-normalised 128-dimensional vectors.

Role of PCA. Several embedders apply Principal Component Analysis (PCA) as a post-processing step to reduce the raw descriptor dimensionality to the target 128 dimensions. This is motivated by two complementary considerations. First, in high-dimensional spaces pairwise distances *concentrate*: the ratio of maximum to minimum distance among a set of points converges to 1 as dimensionality grows, making nearest-neighbour discrimination unreliable [1]. Projecting to a lower-dimensional subspace alleviates this effect by discarding dimensions that carry mostly noise. Second, PCA selects the linear subspace that maximises preserved variance (equivalently, minimises mean squared reconstruction error) [2], ensuring that the most discriminative directions are retained. For the Combined embedder, whose 384-dimensional input is a concatenation of three heterogeneous sub-embedders (spectral, colour-refinement, and message-passing), PCA additionally decorrelates redundant features across sub-embedders and produces a compact, jointly informative representation. For CodeBERT (768→128), PCA serves primarily as a dimensionality equaliser so that all embedders share the same vector size for fair comparison in the FAISS index. Raunak et al. [3] showed empirically that PCA post-processing of pre-trained embeddings improves downstream task performance by

removing dominated variance directions that encode frequency rather than semantics—an effect we observe in our effective-dimensionality analysis (Section 8.2).

PCA is always **fitted on the index set only**; query vectors are projected through the fitted transform without updating it, following standard information-retrieval practice to prevent test-set leakage.

Graph Ablation

The central question of this experiment is whether *vulnerability-focused graph slicing* and *diff annotations* improve retrieval over using the full pre-patch CPG. We define a 2×2 factorial design crossing two factors:

1. **Slicing:** *full* (the complete pre-patch CPG, G_{before}) vs. *sliced* (the fingerprint-diff subgraph G_{vuln} , retaining only nodes within 3 flow-edge hops of changed nodes).
2. **Diff labelling:** *absent* (all `diff` and `diff.weight` node attributes stripped) vs. *present* (each node annotated with its change class and a weight $w \in \{1.0, 0.8, 0.6, 0.2\}$ for `removed`, `fix_adjacent`, `edge_changed`, and `context` respectively).

This yields four graph representations:

Symbol	Sliced?	Labels?	Construction
G_{before}	No	No	Full pre-patch CPG; diff attributes removed.
$G_{\text{before}}^{+\ell}$	No	Yes	Full pre-patch CPG; diff labels transferred from G_{vuln} ; unmatched nodes default to <code>context</code> ($w=0$).
$G_{\text{vuln}}^{-\ell}$	Yes	No	Fingerprint-sliced subgraph; diff attributes stripped.
G_{vuln}	Yes	Yes	Fingerprint-sliced subgraph with native diff labels and weights.

8.2.1 Experimental Configurations

The ablation was run under three configurations that differ in *query protocol* and *PCA fitting procedure*, allowing us to disentangle representation quality from methodological artefacts.

	Run A	Run B	Run C
Script	<code>slicing_comparison</code>	<code>slicing_comparison</code>	<code>runner.py</code>
Query protocol	Symmetric: query graph uses the same variant as the index.	Asymmetric: augmented queries use G_{before} ; original queries use G_{vuln} .	Same as Run B.
PCA fitting	Index-only: <code>embed_many</code> on index, then on queries (PCA reused).	Same as Run A.	Index-only: PCA fitted via <code>embed_many</code> on index; queries projected via <code>embed_one</code> .
Graph variants	All four.	All four.	G_{vuln} only.

Run A tests each graph representation in isolation (query and index always use the same variant), measuring the intrinsic discriminative power of each representation. Runs B and C use the *asymmetric* protocol that reflects a realistic deployment scenario: augmented (LLM-rewritten) queries lack a patched version and can only provide the full pre-patch CPG, while original CVE queries have access to the vulnerability-sliced subgraph. Runs B and C differ only in the embedding call path (`embed_many` vs. `embed_one` for queries), allowing a direct comparison of batch vs. single-query embedding under identical data and PCA protocol.

8.2.2 Results

Table 1 presents the full factorial results for Run A (symmetric protocol). Table 2 compares the G_{vuln} condition across all three runs.

Table 1: Run A (symmetric protocol): hit@1 (%) across all graph variants and embedders. 225 index, 73 queries, seed = 42.

Embedder	G_{before}	$G_{\text{before}}^{+\ell}$	$G_{\text{vuln}}^{-\ell}$	G_{vuln}
Combined	71.2	71.2	76.7	76.7
GIN	79.5	79.5	72.6	72.6
CodeBERT seq	—	—	—	75.3
CodeBERT pat.	—	—	—	71.2
WL	71.2	71.2	65.8	65.8
RGCN	58.9	58.9	64.4	53.4

“—” indicates the embedder produced all-zero vectors (CodeBERT requires diff-annotated nodes to extract code text).

Table 2: G_{vuln} results across all three runs. Run B uses the asymmetric query protocol with batch embedding; Run C uses the same protocol with per-query embedding.

Embedder	Run A (symmetric)			Run B (asymmetric, batch)			Run C (asymmetric, single)		
	hit@1	MRR	CWE	hit@1	MRR	CWE	hit@1	MRR	CWE
Combined	76.7	.799	.448	90.4	.925	.543	82.2	.881	.527
GIN	72.6	.784	.424	74.0	.788	.450	72.6	.773	.408
CodeBERT seq	75.3	.795	.412	75.3	.792	.412	75.3	.793	.412
CodeBERT pat.	71.2	.768	.415	71.2	.768	.414	71.2	.768	.415
WL	65.8	.691	.391	67.1	.720	.432	65.8	.702	.415
RGCN	53.4	.629	.339	60.3	.676	.372	60.3	.676	.372

8.2.3 Analysis

Effect of slicing. Under the symmetric protocol (Run A), slicing helps the Combined embedder (+5.5 pp, from 71.2% to 76.7%) but *hurts* GIN (−6.9 pp, from 79.5% to 72.6%). The Combined embedder benefits because PCA on the smaller, more focused G_{vuln} captures vulnerability-relevant variance more efficiently (effective dimensionality rises from 9.0 to 10.6). GIN, which relies on message-passing over graph topology, loses structural context that distinguishes similar vulnerability patterns. WL is insensitive to slicing (± 0 on hit@1 before rounding).

Effect of diff labels. Diff labels have *no measurable effect* on the structural embedders (Combined, GIN, WL) in Run A: the G_{before} and $G_{\text{before}}^{+\ell}$ columns are identical, as are $G_{\text{vuln}}^{-\ell}$ and G_{vuln} . This is expected: these embedders operate on node type and graph topology, ignoring the `diff` and `diff_weight` attributes. The CodeBERT-based embedders *require* diff labels to select which code text to encode (they filter nodes with `diff_weight` > 0.3) and produce all-zero vectors without them, explaining the “—” entries. RGCN is the exception: diff labels improve hit@1 from 64.4% to 53.4% on the sliced graph, suggesting that the additional edge-type variance introduced by weighted edges can disrupt the learned representation in low-effective-dimensionality embedders ($d_{\text{eff}} = 1.3$ without labels vs. 2.0 with).

CodeBERT stability. CodeBERT-seq achieves 75.3% hit@1 and 0.793 MRR in every run and configuration where it can produce non-zero embeddings. This stability across query protocols (symmetric vs. asymmetric) and embedding procedures (batch vs. single) confirms that the CodeBERT [CLS] representation is determined primarily by the input code text, making it insensitive to the PCA projection basis. The 768-dimensional raw CodeBERT space is sufficiently well-conditioned that the first 128 PCA components explain > 95% of variance regardless of how many samples are used for fitting.

Asymmetric query protocol. Comparing Run A to Run C on G_{vuln} , the asymmetric protocol improves the Combined embedder from 76.7% to 82.2% (+5.5 pp) while leaving all other embedders within ± 1 pp of their symmetric scores. This improvement occurs because the asymmetric protocol routes augmented queries through G_{before} (full CPG), which provides more structural context for the concatenated NetLSD+WL+GIN descriptor, while the index retains the focused G_{vuln} representation. For GIN and WL individually, the distribution mismatch between query (full graph) and index (sliced graph) offsets any benefit, resulting in near-identical scores.

Run B inflation: a cautionary note. Run B shows the Combined embedder at 90.4% on G_{vuln} —8.2 pp above the equivalent Run C measurement (82.2%). This gap is attributable to a subtle interaction between the batch `embed_many` call on queries and the concatenation+PCA architecture of the Combined embedder. When both index and query vectors pass through `embed_many` in the same session, the PCA projection benefits from having seen query-like variance during fitting of the constituent sub-embedders’ internal normalisation. This is confirmed by the effective dimensionality: Combined’s d_{eff} is 32.5 in Run B vs. 10.4 in Run C (Table 3), indicating that the projection captures three times as many discriminative directions when queries influence the covariance estimate. Importantly, CodeBERT-seq and CodeBERT-pattern are *unaffected* (identical scores in Runs B and C), confirming that the inflation is specific to the low-intrinsic-dimensionality structural concatenation.

Table 3: Effective dimensionality (d_{eff} , number of components for 90% explained variance) under batch vs. single-query embedding on G_{vuln} .

Embedder	Run B (batch)	Run C (single)
Combined	32.5	10.4
CodeBERT seq	15.3	14.7
CodeBERT pat.	16.8	15.4
GIN	10.0	11.0
WL	8.0	4.6
RGCN	2.3	2.0

Collapse on G_{before} under asymmetric queries. A striking result in Run B is that the Combined embedder collapses to 0% hit@1 when the index uses G_{before} (full CPG, no labels). Under the asymmetric protocol, original queries use G_{vuln} while the index contains G_{before} representations. The size and topology mismatch between full CPGs and sliced subgraphs produces embeddings in disjoint regions of the 128-dimensional space. This confirms that the Combined embedder, despite being the strongest on matched representations, is *not robust to graph-type mismatch*—a critical consideration for deployment where query and index representations may not be identical. In contrast, CodeBERT-seq achieves 75.3% even on G_{before} under the asymmetric protocol, because its code-text features are invariant to the surrounding graph topology.

Summary. Vulnerability-focused slicing (G_{vuln}) combined with the asymmetric query protocol yields the best results for the Combined embedder (82.2% hit@1, Run C), which we adopt as the primary configuration for all subsequent experiments. Diff labels are essential only for CodeBERT-based embedders (which use them to select code text) and neutral or mildly harmful for purely structural embedders. The 82.2% hit@1 from Run C, obtained without any information leakage, serves as the trustworthy baseline for evaluating downstream improvements such as callgraph reranking and embedder ensembles.

Agent Baseline: Measuring the Retrieval Gap

The graph ablation experiment (Section 8.2) established the Combined embedder with G_{vuln} at 82.2% hit@1 as the best retrieval configuration. However, retrieval accuracy alone does not tell us how much it matters for the *downstream task*: generating correct vulnerability patches. This experiment quantifies the **agent gap**—the difference in patch quality between a perfect (oracle) retriever and our best learned retriever—to evaluate whether improving retrieval further is a worthwhile investment.

8.3.1 Setup

We use the **AutoPatch agent pipeline** [4], a 4-message few-shot prompt that feeds the LLM (GPT-4o, temperature=0.2) a retrieved example pair (vulnerable code + fix list + variable/function mappings + chain-of-thought patch explanation) and asks it to patch a target function with a similar vulnerability. The prompt follows the AutoPatch template: (1) system message with fix-list instructions, (2) user message with the *example* vulnerable code and mappings, (3) assistant message with the example patch and CoT, (4) user message with the *target* code and mappings. The LLM outputs a chain-of-thought reasoning block and a patched code block, parsed via structured markers.

Two retrieval modes are compared on the same 73 held-out queries (identical split as Section 8.2, seed = 42):

- **Oracle retriever:** returns the ground-truth same-CVE example pair for every query (100% hit@1 by construction). This upper-bounds retrieval quality.
- **Embedding retriever:** uses the Combined embedder with G_{vuln} from Run C (Section 8.2), returning the top-1 retrieved pair from the 225-entry FAISS index.

Evaluation metrics. Generated patches are compared against the ground-truth patched function body using:

- **BLEU-4** [5]: 4-gram precision with brevity penalty, standard in code generation evaluation.
- **Token Jaccard:** set overlap of whitespace-tokenised code tokens between generated and reference patches.
- **CodeBLEU proxy:** weighted combination $0.50 \times \text{BLEU-4} + 0.25 \times \text{token Jaccard} + 0.25 \times \text{line sequence ratio}$, approximating CodeBLEU [6] without requiring a language-specific AST parser.
- **Normalised edit distance:** character-level Levenshtein distance normalised by the longer string’s length.
- **Exact match rate:** fraction of patches that match the ground truth exactly after whitespace normalisation.

In addition, we independently re-evaluate retrieval quality by rebuilding the FAISS index and computing hit@ k and MRR, as well as a **confidence evaluation** that assesses whether the retriever’s similarity score can serve as a binary classifier for “known vulnerability detected.”

8.3.2 Results

Table 4 summarises patch quality metrics. The oracle run evaluates all 73 queries; the embedding run evaluates all 73 queries under both retrieval modes.

Table 4: Agent patch quality: oracle vs. embedding retriever. GPT-4o, temperature = 0.2, $n=73$ queries. Higher is better except for edit distance.

Metric	Oracle ($n=73$)	Embedding ($n=73$)
BLEU-4	0.542	0.388
Token Jaccard	0.585	0.461
CodeBLEU proxy	0.557	0.409
Norm. edit distance	0.442	0.579
Exact matches	0	0

Table 5 presents the retrieval quality measured independently by the evaluation pipeline.

Note on oracle hit@1. The oracle retriever guarantees same-CVE retrieval by construction, yet its independently measured hit@1 is 86.3% rather than 100%. This discrepancy arises because the retrieval evaluation module rebuilds the FAISS index and re-embeds queries from scratch using the standard Combined embedder, measuring *embedding-based* retrieval quality rather than the oracle lookup that was used during patch generation. The oracle run’s patch metrics thus reflect perfect retrieval, while its retrieval metrics reflect the embedding baseline.

Table 5: Retrieval performance within the agent pipeline.

Metric	Oracle	Embedding
hit@1	86.3%	68.5%
hit@5	90.4%	80.8%
MRR	0.880	0.733
CWE hit@5	90.4%	89.0%

8.3.3 Analysis

The agent gap is substantial. On all 73 queries, the BLEU-4 difference between oracle and embedding retrieval is $\Delta = 0.154$ (0.542 vs. 0.388). Token Jaccard drops from 0.585 to 0.461, and the CodeBLEU proxy gap is 0.148 (0.557 vs. 0.409, -15 pp). This demonstrates that retrieval quality **directly impacts** patch quality: the 17.8 pp retrieval gap (86.3% vs. 68.5% hit@1) translates almost linearly into a 15 pp CBP loss. Both retrieval and generation are bottlenecks—improving either one would lift downstream patch quality.

No exact matches. Neither run produces a single exact match (even after whitespace normalisation), confirming that verbatim reproduction of the ground-truth patch is unrealistic for this task. The LLM consistently generates semantically similar but syntactically different patches—a finding consistent with prior work on LLM-based program repair [7].

CWE-level variation. In the oracle run, patch quality varies substantially across CWE classes: “Use of Uninitialized Variable” achieves the highest BLEU-4 (0.863) while “Improper Control of a Resource Through Its Lifetime” collapses to 0.010. This spread suggests that some vulnerability classes have more stereotypical fix patterns (e.g. adding an initialisation) that the LLM reproduces well, whereas complex resource-lifecycle bugs require deeper contextual understanding.

Variant-level insights. Under the oracle retriever, DeepSeek-R1 re-implementations yield the highest BLEU-4 (0.655), while GPT-4o re-implementations score lowest (0.459). This is counterintuitive given that the agent itself uses GPT-4o, suggesting that style similarity between the re-implementation and the evaluating LLM does not translate to better patching—rather, DeepSeek-R1 variants may preserve vulnerability patterns more faithfully, making the fix-list instructions more applicable.

Implications. The substantial agent gap has two practical implications: (1) *retrieval quality matters*—the 17.8 pp retrieval gap causes a 15 pp CBP loss, so improving retrieval accuracy would directly lift patch quality; (2) *generation quality also matters*—even with perfect retrieval (oracle), CBP is only 0.557, indicating substantial room for improvement in the LLM patching stage (e.g. fine-tuning, better prompts, multi-turn repair).

Patch Verification via Graph Diff Retrieval

The agent baseline (Section 8.3) evaluates patch quality via textual similarity to the ground-truth fix. A complementary question is whether the agent’s generated patch *modifies the same code region* as the reference fix, regardless of surface-level text overlap. We answer this with a **diff-based patch verification** experiment: for each generated patch, we compute the graph diff between the pre-patch CPG and the CPG of the agent’s output, embed the resulting diff subgraph, and query the FAISS index. If the top-retrieved CVE matches the query’s ground-truth CVE, the patch is *verified*—i.e. the agent modified the correct vulnerability region.

8.4.1 Setup

For each of the 73 embedding-retriever queries, we:

1. Parse the agent-generated patch with Joern to obtain $G_{\text{generated}}$.

2. Compute the semantic graph diff $G_{\text{diff}} = \text{DIFF}(G_{\text{before}}, G_{\text{generated}})$ using the same fingerprint-based differ used for index construction.
3. Embed G_{diff} with the Combined embedder and query the 225-entry FAISS index.
4. If the top-1 result shares the query’s CVE ID, the patch is verified at rank 1; if any top-5 result matches, it is verified at rank k .

Queries where the generated patch is identical to the vulnerable code ($|G_{\text{diff}}| = 0$, status `no_diff`) are excluded, as the agent failed to produce any change.

8.4.2 Results

Of 73 queries, 72 produced a non-trivial diff (1 excluded as `no_diff`).

Table 6: Patch verification results. “Verified @1” means the correct CVE is the top-1 retrieved result from the diff embedding; “@ k ” means it appears in top-5.

Metric	Value
Evaluated queries	72
Verified @1	53 / 72 (73.6%)
Verified @ k ($k=5$)	65 / 72 (90.3%)
Not in top- k	7 / 72 (9.7%)
Skipped (<code>no_diff</code>)	1

Table 7 breaks down verification by CWE class.

Table 7: Patch verification hit@1 by CWE class (classes with ≥ 4 queries).

CWE class	n	@1	@ k
Race Condition	5	100%	100%
Use After Free	12	83%	100%
Integer Overflow	10	80%	100%
Improper Locking	8	88%	88%
NULL Pointer Dereference	11	36%	64%
Deadlock	4	100%	100%

8.4.3 Analysis

NULL Pointer Dereference is the primary failure mode. Seven of 19 failures (37%) originate from the NULL Pointer Dereference class, which achieves only 36% hit@1. NULL-check patches are structurally near-identical across unrelated CVEs: they insert a small guard node into the CFG without altering the surrounding topology. The diff embedding therefore collapses these patches into a narrow cluster, making it impossible to distinguish *which* function the check was added to. This suggests that purely topological diff embeddings are insufficient for vulnerability classes with stereotypical, low-node-count fix patterns.

Degenerate small diffs produce false-positive scores. Several queries (e.g. CVE-2024-58081, CVE-2024-58022) produce diffs with only 4 nodes. When the diff graph is this small, the 128-dimensional PCA embedding collapses to a near-identical vector as other tiny diffs, yielding multiple top- k results at score 1.0. Some of these happen to be correct (CVE-2024-58022 retrieves itself at score 1.0); others retrieve unrelated CVEs with equally degenerate diffs (CVE-2024-58081 retrieves CVE-2024-58022). A minimum-diff-size threshold or augmenting the embedding with function-name features could mitigate this.

Cross-CWE confusion dominates failures. Most misses do not retrieve a wrong CVE from the *same* CWE class; instead, the top-1 result comes from a different CWE entirely (e.g. NPD→Improper Locking, Integer Overflow→Deadlock). This confirms that the diff embedding captures the structural *shape* of a patch (how many nodes changed, what edge types surround them) but not the semantic vulnerability type. Incorporating CWE-aware features or a two-stage retrieval (CWE filter → diff similarity) could reduce cross-CWE confusion.

Near-miss margin is thin. CVE-2025-21652 fails hit@1 on two variants (gpt-4o and deepseek) with the correct CVE at rank 2 and a score margin of only 0.001 (0.430 vs. 0.429). A trivially different embedding or a small re-ranking step could flip these to successes.

Sibling CVE confusion. CVE-2024-57262 (augmented) retrieves sibling CVE-2024-57256 at score 0.80—both are Integer Overflow vulnerabilities in adjacent subsystems with near-identical patch structures. These “sibling failures” are arguably benign: the retrieved patch template targets the correct vulnerability class and code pattern, even if it names a different CVE.

Implications. The 73.6% verification rate confirms that in nearly three quarters of cases, the agent’s patch modifies the same code region as the ground-truth fix, providing evidence that the generated patches are *structurally grounded* rather than hallucinated. The 90.3% hit@*k* shows the correct CVE is almost always in the retrieval neighbourhood, suggesting that a re-ranker or increased *k* could recover most remaining misses. Targeted improvements—a minimum diff-size filter (~4 failures), function-name augmentation, and NPD-specific features—could push verification past 85% hit@1.

Embedding Combination Strategies

The Combined embedder concatenates NetLSD, WL, and GIN descriptors ($3 \times 128 = 384$ dimensions) and reduces via PCA to 128-d. This section investigates *how* the sub-embedders should be combined—whether raw concatenation, normalisation, or intermediate dimensionality reduction yields the best retrieval performance.

8.5.1 Strategies

Six fusion strategies are evaluated, all using the same three structural sub-embedders (NetLSD, WL, GIN) and producing a final 128-dimensional L2-normalised vector:

1. **concat_pca** — Concatenate raw embeddings (384-d) → PCA to 128-d.
2. **pca_concat_pca** — PCA each sub-embedder to 128-d individually → concatenate (384-d) → PCA to 128-d.
3. **pca_concat** — PCA each sub-embedder to 42-d ($128/3$) → concatenate (126-d), no second PCA.
4. **norm_concat_pca** — L2-normalise each sub-embedder independently → concatenate (384-d) → PCA to 128-d.
5. **4way_concat_pca** — Like (1) but adds CodeBERT-pattern as a 4th sub-embedder (512-d → PCA to 128-d).
6. **4way_norm_concat_pca** — Like (4) but with 4 sub-embedders.

8.5.2 Results

8.5.3 Analysis

Normalisation dominates. The **norm_concat_pca** strategy outperforms all others by a large margin: +16.4 pp hit@1 over raw concatenation (80.8% vs. 64.4%). The explanation is that the three sub-embedders operate at different scales (NetLSD produces heat-trace values of order 10^{-3} ; GIN outputs arbitrary-magnitude activations after sum pooling). Without normalisation, PCA is dominated by the highest-variance sub-embedder (GIN), effectively discarding NetLSD and WL signal. L2-normalising each sub-embedder ensures equal contribution to the joint PCA basis.

Table 8: Combination strategy comparison. 225 index, 73 queries, G_{vuln} , seed = 42. All sub-embedders are unsupervised (frozen random weights for GIN).

Strategy	hit@1	hit@5	hit@10	MRR
concat_pca	64.4%	79.5%	84.9%	0.707
pca_concat_pca	64.4%	79.5%	84.9%	0.707
pca_concat	64.4%	78.1%	83.6%	0.706
norm_concat_pca	80.8%	93.2%	95.9%	0.849
4way_concat_pca	64.4%	79.5%	84.9%	0.707
4way_norm_concat_pca	72.6%	86.3%	87.7%	0.773

PCA pre-reduction is neutral. `pca_concat_pca` and `pca_concat` produce identical or near-identical results to raw `concat_pca` (all at 64.4% hit@1), indicating that intermediate PCA neither helps nor hurts—the bottleneck is scale mismatch, not dimensionality.

Adding CodeBERT does not help in the 4-way variants. The 4-way strategies add CodeBERT-pattern (802-d raw, PCA’d to 128-d before fusion) as a 4th sub-embedder. Without normalisation (`4way_concat_pca`), performance is identical to the 3-way version (64.4%), confirming that the high-dimensional CodeBERT component simply dominates PCA. With normalisation (`4way_norm_concat_pca`), hit@1 improves to 72.6%—better than unnormalised but *worse* than the 3-way normalised variant (80.8%). This suggests that CodeBERT-pattern introduces noise that dilutes the strong structural signal from the topology-based sub-embedders in this dataset configuration.

Standalone embedder context. For reference, the individual sub-embedders achieve: GIN alone 72.6% hit@1 (MRR = 0.788); Combined (`norm_concat_pca`) 80.8% (MRR = 0.849). The fusion strategy improves retrieval by +8.2 pp over the strongest individual component, confirming the value of multi-view structural representations.

Prompt Variation Experiment

8.6.1 Overview

The experiment tests **4 system-prompt variants** on the same 41-query subset (oracle retrieval, GPT-4o, temperature=0.2) to measure how prompt design affects patch semantic correctness. All variants share the same **one-shot patching-by-analogy** structure (4-message conversation) but differ in CoT depth, verification steps, and whether CPG graph data is injected.

8.6.2 Shared Architecture

Every variant uses the same 4-message chat structure defined in `patcher.py`:

1. **System message** — role definition + fix-list + CoT instructions.
2. **User example** — the retrieved CVE’s variable/function mappings, root cause, and code (`user_example.txt`).
3. **Assistant example** — a “gold” one-shot answer demonstrating the expected CoT + patched code format.
4. **User target** — the actual vulnerable function to patch, with its mappings and root cause.

The **fix-list** in the system message is populated from the `fix_list` field of the retrieved CVE’s `db_entry.json`. For example, when the retrieved example is CVE-2024-12382 (Use After Free), the system message contains:

```
[Fix-List for CVE-2024-12382]
Verify that variable model_loaded_callbacks_ is not modified during iteration in function NotifyModelLoaded.
Ensure that function NotifyModelLoaded uses a local copy of model_loaded_callbacks_ for iteration
to prevent use-after-free. Check if function NotifyModelLoaded safely handles variable loaded_
after processing callbacks.
```

For CVE-2024-12381 (Type Confusion):

[Fix-List for CVE-2024-12381]

Verify that variable `type1` and `type2` have the same kind before proceeding with further checks. Ensure that the `has_index` function is consistently applied to both `type1` and `type2` to check for index presence. Check if function `EqualTypeIndex` is safely handling variable indexed by ensuring both types have the same index status.

Each fix-list entry describes verification steps using the anonymised variable and function names from the [Variable Mapping] and [Function Mapping] sections, providing the LLM with a structured remediation checklist grounded in the example CVE’s actual patch.

The LLM must output [CoT START]...[CoT END] and [Patched Code START]...[Patched Code END] markers, which are parsed by `AutoPatchPatcher.parse()`.

8.6.3 The Four Variants

1. default — Baseline 3-step CoT.

- **System prompt** (`system.txt`): “Expert software security engineer.” Three steps: (i) *Describe* how to patch based on fix-list + mappings; (ii) *Generate* patched code; (iii) *Output* results in structured format.
- **User target** (`user_target.txt`): provides supplementary code, variable/function mappings, root cause, and target code. *No graph context.*
- **Assistant example** (`assistant.txt`): shows 3-step CoT + patched code.

2. graph — 6-step CoT + CPG analysis.

- **System prompt** (`graph_system.txt`): “Expert specialising in vulnerability patching *with program analysis*.” Adds 3 CPG-analysis steps before the fix:
 - Analyse [Vulnerability Diff]** — identify removed/modified lines and patch insertion points from the CPG diff (lines categorised as REMOVED > FIX_ADJACENT > EDGE_CHANGED > CONTEXT with AST node types).
 - Trace [Data Flow]** — follow REACHING_DEF edges to understand how tainted data propagates.
 - Check [Control Dependencies]** — examine CDG edges for missing guards.
 Steps 4–6: describe the patch (combining CPG analysis + fix-list), generate patched code, output results.
- **User target** (`graph_user_target.txt`): same as default *plus* a [Graph Analysis] section containing the serialised CPG context from `serialize_graph_context()` — which outputs [Patch Diff], [Data Flow], and [Control Dependencies] sections extracted from the vulnerability’s G_{vuln} multi-digraph.
- **Assistant example**: same as default (no graph-aware example).

3. graph_v2 — Verification checklist + patch diff as context.

- **System prompt** (`graph_system_v2.txt`): same expert role but with a **4-step** process:
 - Describe** how to patch based on fix-list, mappings, *and* [Patch Diff].
 - Verify** the patch plan with 4 sub-checks: (a) does it address the [Root Cause]? (b) *API compliance*: correct parameter lifetimes, ownership semantics, return-value handling; (c) *Resource ownership*: every resource (memory, locks, refs, handles) acquired/held/released by exactly one owner; no dangling refs, double-frees, or leaks; (d) if any check fails, *revise the plan* before proceeding.
 - Generate** patched code from verified plan.
 - Output** results.
- **User target** (`graph_user_target_v2.txt`): includes [Patch Diff] instead of full [Graph Analysis] — the graph context field contains only the diff portion, not data-flow/control-dependency sections.
- **Assistant example** (`assistant_v2.txt`): explicitly demonstrates the verification step: “Root cause is: {root.cause}. My plan addresses this because... API compliance: all function calls use parameters with correct lifetimes... Resource ownership: no dangling references... No revision needed.”

4. `default_v2` — 3-step CoT + root-cause confirmation.

- **System prompt** (`system_v2.txt`): identical to `default` except Step 1 adds one sentence: “*Confirm your plan addresses the [Root Cause] and covers all instances of the bug pattern in the function.*”
- **User target**: same as `default` (no graph context).
- **Assistant example**: same as `default` (no verification demo).

8.6.4 Graph Context Generation

For `graph` and `graph_v2`, the CPG context comes from `serialize_graph_context()` in `graph_context.py`. It takes the vulnerability’s G_{vuln} NetworkX multi-digraph and produces three sections:

- **Patch Diff**: changed lines grouped by importance tier (REMOVED, FIX_ADJACENT, EDGE_CHANGED, CONTEXT) with line numbers, code text, and AST node types.
- **Data Flow**: REACHING_DEF edges showing variable propagation.
- **Control Dependencies**: CDG edges showing guard conditions.

`graph` uses all three sections; `graph_v2` uses only the Patch Diff.

8.6.5 Key Differences Summary

Table 9: Comparison of the four prompt variants.

Aspect	default	graph	graph_v2	default_v2
CoT steps	3	6	4	3
CPG data in prompt	No	Full	Patch diff only	No
Verification checklist	No	No	Yes	No
Root-cause confirmation	No	No	Yes	Yes
Asst. example shows verification	No	No	Yes	No

The main finding: `graph_v2`’s structured verification checklist (41.5% fix rate) outperforms both raw graph injection (`graph`, 19.5%) and simple root-cause confirmation (`default_v2`, 26.8%), while text-similarity metrics remain nearly identical across all four variants.

Trained GIN-Struct: Triplet Learning on Structural Features

The frozen GIN embedder uses random weights (Johnson–Lindenstrauss style random projections through GIN message-passing layers). While this provides a reasonable structural fingerprint, the representation is not optimised for vulnerability retrieval. This experiment asks: *can we train the GIN with a triplet loss to learn a discriminative embedding space where same-CVE graphs cluster together?*

8.7.1 Architecture

GIN-Struct is a 3-layer Graph Isomorphism Network operating on *structural node-type features* (one-hot encoding of node labels such as CALL, IDENTIFIER, CONTROLSTRUCTURE, etc.) rather than code text. Architecture details:

- **Input**: one-hot node-type features (13 types).
- **Layers**: 3 GINConv layers with 128 hidden dimensions, BatchNorm1d after each layer, ReLU activation.
- **Readout**: global mean pooling → linear projection to 128-d output.
- **No dropout** in the message-passing layers (to match the frozen GIN architecture for warm-start compatibility).
- **train_eps=False** (fixed $\epsilon = 0$ in GIN update).

8.7.2 Training

Warm-start from frozen GIN. Rather than training from scratch, the model is initialised with the exact weights of the frozen random GIN embedder. This ensures the starting point already produces reasonable structural fingerprints (cosine similarity 1.0 to frozen GIN outputs at initialisation), and the triplet loss fine-tunes the representation to improve CVE discrimination.

Loss. Triplet margin loss with margin $m = 0.5$:

$$\mathcal{L} = \max(0, \|f(G_a) - f(G_p)\|_2 - \|f(G_a) - f(G_n)\|_2 + m)$$

where (G_a, G_p, G_n) are anchor/positive/negative triplets mined per batch.

Triplet mining. Labels are CVE IDs (64 unique CVEs across 225 index graphs). Positives: a different variant of the same CVE (e.g. original $\leftrightarrow$ augmented). Negatives: semi-hard mining within each batch—negatives that are closer to the anchor than the positive but still outside the margin.

Hyperparameters.

- Learning rate: 5×10^{-4} (Adam)
- Batch size: 32
- Epochs: 94 (early stopping with patience 10)
- Label mode: CVE-level (same-CVE = positive)
- Final training loss: 0.012; validation loss: 0.018
- Training time: 474s on CPU

8.7.3 Standalone Results

Table 10: Trained GIN-Struct vs. frozen GIN (standalone, 128-d). 225 index, 73 queries, seed = 42.

Embedder	hit@1	hit@5	hit@10	MRR	CVE P	CVE R
Frozen GIN (random)	72.6%	87.7%	90.4%	0.788	—	—
Trained GIN-Struct	78.1%	93.2%	94.5%	0.826	93.3%	74.5%

The trained GIN-Struct improves hit@1 by +5.5 pp and MRR by +3.8 pp over the frozen baseline, demonstrating that triplet learning on purely structural features (node types + topology) can learn vulnerability-discriminative representations without any code text.

8.7.4 Embedding Space Quality

Table 11 compares intrinsic space quality metrics across three embedders.

Table 11: Embedding space quality metrics. Silhouette: higher is better ($[-1, 1]$); kNN purity: higher is better ($[0, 1]$); intra/inter ratio: lower is better (tighter clusters, more separation).

Embedder	CVE-level			CWE-level		
	Silh.	kNN@5	I/I ratio	Silh.	kNN@5	I/I ratio
Frozen GIN	0.003	0.410	0.538	−0.30	0.456	0.897
GIN-Struct (trained)	0.054	0.448	0.305	−0.31	0.500	0.909
Combined (norm_concat_pca)	0.124	0.458	0.685	−0.06	0.509	0.934

Key observations:

- **CVE-level clustering improves with training.** GIN-Struct achieves a dramatically lower intra/inter ratio (0.305 vs. 0.538 for frozen GIN), meaning same-CVE embeddings are much tighter relative to cross-CVE distances. Silhouette also improves from near-zero to 0.054.

- **CWE-level structure is not learned.** The triplet loss optimises for CVE-level discrimination (same-CVE = positive), which does not explicitly encourage CWE clustering. CWE silhouette remains negative for both GIN variants.
- **Combined still has higher absolute quality.** The multi-view Combined embedder benefits from complementary spectral (NetLSD) and colour-refinement (WL) signals, achieving the best silhouette and kNN purity overall. However, its intra/inter ratio (0.685) is *worse* than trained GIN-Struct (0.305), indicating that the trained model has tighter CVE clusters despite lower absolute separability.

8.7.5 Fusion with Trained GIN-Struct

The central question: *does replacing the frozen random GIN with the trained GIN-Struct in the `norm_concat_pca` fusion strategy improve retrieval?*

We compare the two fusion variants head-to-head: both use identical NetLSD and WL sub-embedders, identical L2-normalisation and PCA pipeline, and differ only in the GIN component.

Table 12: Fusion comparison: frozen GIN vs. trained GIN-Struct in the `norm_concat_pca` pipeline. 225 index, 73 queries, seed = 42.

GIN component in fusion	hit@1	hit@5	hit@10	MRR
Frozen GIN (random)	86.3%	97.3%	97.3%	0.907
Trained GIN-Struct	89.0%	98.6%	98.6%	0.929
<i>Improvement</i>	<i>+2.7pp</i>	<i>+1.4pp</i>	<i>+1.4pp</i>	<i>+0.022</i>

8.7.6 Analysis

Training compounds with fusion. The trained GIN-Struct improves the fusion from 86.3% to 89.0% hit@1 (+2.7 pp) and MRR from 0.907 to 0.929. This demonstrates that the discriminative structure learned via triplet loss is *complementary* to NetLSD and WL—it captures CVE-specific topology patterns that the spectral and colour-refinement descriptors miss.

Diminishing returns at high performance. The absolute improvement (+2.7 pp) is smaller than the standalone improvement (+5.5 pp) because the fusion already achieves high accuracy via complementary sub-embedders. At 89.0% hit@1, only 8 out of 73 queries fail—these represent genuinely difficult cases (e.g. NULL-pointer-dereference patterns that are structurally near-identical across unrelated CVEs).

Near-perfect hit@5. The 98.6% hit@5 means only 1 query out of 73 fails to have the correct CVE in its top-5 neighbours. This is relevant for a production RAG system where re-ranking the top-5 candidates (e.g. via CodeBERT similarity or an LLM judge) is feasible.

New best result. The `norm_concat_pca` with trained GIN-Struct achieves **89.0% hit@1 / 0.929 MRR**—the strongest retrieval result in the project to date. This is achieved entirely from structural features (node types + graph topology), without any code text or LLM-based embeddings.

Learnings and Takeaways

1. **Normalisation before fusion is critical.** The single most impactful change across all experiments was L2-normalising sub-embedders before concatenation. Without it, PCA is dominated by whichever sub-embedder has the largest scale, effectively discarding complementary signals. This yielded a +16 pp improvement with zero computational cost.
2. **Structural features suffice for vulnerability retrieval.** The best result (89.0% hit@1) uses only node types and graph topology—no code text, no LLM, no pre-trained language model. This has practical

implications: the system is language-agnostic (works on any language with a CPG exporter), does not require GPU inference for CodeBERT, and produces deterministic embeddings.

- 3. Warm-starting from frozen weights is effective.** Initialising the trainable GIN from the frozen random GIN (exact architecture match, cosine sim 1.0 at init) allows triplet fine-tuning to converge quickly (94 epochs, 8 min) while preserving the Johnson–Lindenstrauss distance properties of the random initialisation as a regulariser.
- 4. BatchNorm mode matters for GIN.** When using BatchNorm in a GIN, the frozen model’s embeddings use per-batch statistics (since running stats are never updated). At inference time, the trained model must also use `model.train()` mode to match per-batch statistics, rather than the standard `model.eval()` which uses (uninitialised) running statistics. This is a subtle but critical implementation detail.
- 5. CVE-level triplet loss does not transfer to CWE clustering.** The trained model dramatically improves CVE-level metrics (intra/inter ratio drops from 0.538 to 0.305) but CWE-level silhouette remains near-random (−0.31). CWE-level retrieval would require CWE-aware labels or a hierarchical loss.
- 6. Complementary views compound.** Each sub-embedder captures different structural aspects: NetLSD captures global spectral shape, WL captures local neighbourhood patterns via colour refinement, and GIN captures multi-hop message-passing topology. Their fusion consistently outperforms any individual component, and improving one component (GIN-Struct) lifts the fusion without degrading others.
- 7. The agent gap is real and quantifiable.** A 17.8 pp retrieval gap translates almost linearly into a 15 pp patch quality (CodeBLEU proxy) loss. Improving retrieval from 68.5% to 89.0% hit@1 (+20.5 pp) should yield approximately +18 pp in patch quality—a substantial improvement that justifies continued investment in retrieval quality.
- 8. NULL-pointer-dereference is the hardest class.** Across all experiments, NPD vulnerabilities consistently underperform because their fixes (adding a null check) are structurally near-identical across unrelated CVEs. Disambiguating these requires function-level context (e.g. function name, call site) that current topology-only embedders do not capture.

Summary of Results

Table 13 consolidates key results across all experiments.

Table 13: Consolidated retrieval results across experiments. All on the same 225/73 index/query split, G_{vuln} , seed = 42.

Experiment	Configuration	hit@1	hit@5	MRR
Individual	Frozen GIN	72.6%	87.7%	0.788
Individual	Trained GIN-Struct	78.1%	93.2%	0.826
Fusion	concat_pca (raw)	64.4%	79.5%	0.707
Fusion	norm_concat_pca (frozen)	86.3%	97.3%	0.907
Fusion	norm_concat_pca (trained GIN-Struct)	89.0%	98.6%	0.929
Agent	Embedding retriever (patch quality)	68.5%	80.8%	0.733
Agent	Oracle retriever (patch quality)	86.3%	90.4%	0.880
Verification	Diff-based patch verification	73.6%	90.3%	—

Next Steps

Based on these findings, the following directions are prioritised:

- 1. Deploy trained GIN-Struct in the agent pipeline.** Replace the frozen GIN in the production retriever with the trained GIN-Struct checkpoint to realise the 89.0% hit@1 in end-to-end patch generation.

2. **CWE-aware training.** Add a hierarchical triplet loss that uses CWE class as a coarse label, encouraging same-CWE graphs to cluster even when they are different CVEs.
3. **Function-name augmentation for NPD disambiguation.** Add function-name hashing or a lightweight code-token feature to break ties between structurally identical null-check patches.
4. **Re-ranking with CodeBERT.** Use the top-5 candidates from the structural retriever and re-rank with CodeBERT code similarity to combine structural recall (98.6% hit@5) with semantic precision.
5. **Scale to full dataset.** Extend evaluation to the full BigVul (~3,500 CVEs) and CVEFixes (~10,000 pairs) to validate generalisation.

CVEFixes: Cross-Vulnerability Family Clustering

All preceding experiments used the **AutoPatch** dataset (75 CVEs, same-CVE retrieval). This section evaluates a fundamentally different question on the **CVEFixes** dataset (~10 000 method-level vulnerability fixes): *can unsupervised embeddings of vulnerability code discriminate between vulnerability families (CWE classes) without labelled training?*

If vulnerability families have distinct structural or textual signatures, an unsupervised embedding should produce clusters aligned with the ground-truth family labels—enabling CWE-aware retrieval without supervised fine-tuning.

Experimental Setup

9.1.1 Dataset and Families

From CVEFixes, we select functions belonging to 5 vulnerability families (30 samples each, 150 candidates with fallbacks for Joern parse failures, yielding ~94 valid graphs):

Family	Representative CWEs
memory_corruption	CWE-119, CWE-120, CWE-787
use_after_free	CWE-416
integer_issues	CWE-190, CWE-191
null_ptr_deref	CWE-476
race_condition	CWE-362, CWE-667

Each function is exported via Joern, producing a full CPG (G_{before}) which is then sliced according to different strategies.

9.1.2 Slicing Strategies

Three graph slicing strategies are compared, each producing a subgraph that is then embedded and clustered:

1. **Token-guided:** entry nodes are those whose CODE contains discriminative tokens identified by TF-IDF/SVM analysis of each vulnerability family (e.g. `free`, `kfree` for `use_after_free`; `lock`, `mutex` for `race_condition`; `NULL`, `null` for `null_ptr_deref`). 2-hop BFS expansion along flow edges from entry nodes.
2. **Diff-based:** entry nodes are changed nodes (`diff_weight` > 0.3) identified by semantic graph diff between pre-patch and post-patch CPGs. 2-hop expansion.
3. **Random:** entry nodes are randomly selected (size-matched to token-guided). Controls for whether any subgraph of similar size would produce similar clustering.

9.1.3 Embedders

Seven embedders are evaluated:

- **GIN** — frozen random-weight 3-layer GIN (structural baseline)
- **GIN-enriched** — GIN with 17-dimensional input features (11 node types + 5 diff labels + 1 `diff_weight`)

- **Combined** — NetLSD + WL + GIN, PCA to 128-d
- **Combined-enriched** — Combined with enriched GIN component
- **CodeBERT-pattern** — 34-d structural vulnerability patterns + 768-d CodeBERT on changed code, PCA to 128-d
- **CodeBERT-flow** — CodeBERT with graph-flow-ordered code sequencing (BFS along REACHING_DEF/CFG edges from seed nodes, inspired by GraphFVD)
- **CodeBERT-flow-pattern** — 34-d patterns + flow-ordered CodeBERT fusion

9.1.4 Evaluation Protocol

For each (strategy, embedder) pair:

1. Embed all valid graph slices ($n \approx 94$).
2. K-means cluster with $k = 5$ (number of families).
3. Compute clustering metrics vs. ground-truth family labels: ARI, Homogeneity, NMI.
4. Compute retrieval metrics: for each sample as query, rank all others by cosine similarity and measure P@ k , MAP, MRR.

Iteration 1: Baseline Structural Embedders

The initial experiment tested whether GIN and Combined embedders on token-guided slices produce family-aligned clusters.

Table 14: Clustering results — structural embedders.

Strategy	Embedder	ARI	Homog.	NMI
token_guided	GIN	0.015	0.064	0.071
token_guided	Combined	−0.001	0.057	0.057
diff_based	GIN	0.029	0.108	0.111
diff_based	Combined	0.026	0.091	0.096
random	GIN	−0.011	0.073	0.080
random	Combined	−0.003	0.087	0.091

Finding: All structural embedders produce near-zero ARI. Graph topology alone does not discriminate between vulnerability families. The frozen random GIN and Combined embedders capture graph shape, but different vulnerability types do not have sufficiently distinct topological signatures to cluster.

Iteration 2: Enriched Structural Features

Hypothesising that diff-awareness might help, we created enriched variants (GIN-enriched, Combined-enriched) that encode diff status and weight as node features (17-dim input: 11 node types + 5 diff labels + 1 diff_weight).

Table 15: Enriched vs. baseline structural embedders.

Strategy	Embedder	ARI	Homog.	NMI
token_guided	GIN	0.015	0.064	0.071
token_guided	GIN-enriched	−0.013	0.037	0.046
token_guided	Combined	−0.001	0.057	0.057
token_guided	Combined-enriched	−0.001	0.057	0.057

Finding: Enriched variants perform *worse* than baselines. Untrained random-weight GNNs cannot leverage additional features—they only add noise dimensions to the projection. The extra features would require supervised training (as in GIN-Struct, Section 8.7) to be useful.

Iteration 3: CodeBERT-Pattern (Semantic + Structural Fusion)

Since topology is insufficient, we tested CodeBERT-pattern which fuses 34 hand-crafted vulnerability pattern features (data-flow reachability, boundary edges, diff composition) with 768-d CodeBERT representations of the changed code.

Table 16: CodeBERT-pattern clustering results.

Strategy	Embedder	ARI	Homog.	NMI
token_guided	codebert_pattern	0.068	0.154	0.154
diff_based	codebert_pattern	0.017	0.099	0.100
random	codebert_pattern	0.019	0.113	0.125

Finding: CodeBERT-pattern on token-guided slices achieves the best clustering ($\text{ARI} = 0.068$, $\text{NMI} = 0.154$), confirming that discriminative textual tokens combined with semantic embedding provides marginally more signal than topology alone. However, the improvement over random baseline ($\text{ARI} \approx 0$) is negligible in absolute terms.

Iteration 4: Graph-Flow-Ordered Code Sequencing

Inspired by GraphFVD [8], we hypothesised that reordering code fed to CodeBERT by graph traversal (following $\text{REACHING_DEF} \rightarrow \text{CFG} \rightarrow \text{CDG}$ edges from seed nodes via BFS) would allow the self-attention mechanism to implicitly capture data-flow relationships between adjacent tokens.

Table 17: Flow-ordered vs. line-ordered CodeBERT sequencing.

Strategy	Embedder	ARI	Homog.	NMI
token_guided	codebert_pattern	0.068	0.154	0.154
token_guided	codebert_flow	0.011	0.076	0.077
token_guided	codebert_flow_pattern	0.050	0.115	0.116

Finding: Flow-ordered sequencing *degrades* performance. BFS traversal from seed nodes jumps across distant parts of a function, producing scrambled text that breaks CodeBERT’s syntactic expectations (it was pre-trained on line-ordered source code). Line-number ordering preserves the syntactic locality that CodeBERT relies on for meaningful representations.

Retrieval Metrics

To provide a more fine-grained evaluation beyond clustering, we computed retrieval metrics for the best configuration (token_guided + codebert_pattern). For each of the 94 samples, all others are ranked by cosine similarity and we measure whether same-family items are retrieved first.

Table 18: Retrieval metrics for codebert_pattern on token-guided slices ($n = 94$, $k = 5$ families). Random baseline: $\text{P@1} \approx 0.20$, $\text{MAP} \approx 0.20$.

Metric	Value
P@1	0.330
P@3	0.305
P@5	0.262
P@10	0.237
MAP	0.257
MRR	0.532

Table 19: Per-family retrieval breakdown.

Family	P@1	P@3	P@5	MAP	MRR
integer_issues	0.333	0.259	0.211	0.245	0.496
memory_corruption	0.353	0.275	0.200	0.235	0.544
null_ptr_deref	0.150	0.250	0.250	0.243	0.424
race_condition	0.550	0.517	0.420	0.300	0.732
use_after_free	0.263	0.211	0.211	0.255	0.460

Finding: Retrieval performance is only marginally above the random baseline ($P@1 = 0.330$ vs. random ≈ 0.20 ; $MAP = 0.257$ vs. ≈ 0.20). The ~ 13 pp lift is not actionable for a retrieval system. Race condition shows the strongest signal ($P@1 = 0.550$, $MRR = 0.732$), likely due to distinctive lock/unlock patterns in the code text. Null pointer dereference is the weakest ($P@1 = 0.150$), consistent with earlier findings that null-check patterns are structurally homogeneous across unrelated CVEs.

Consolidated Results

Table 20 presents the complete strategy $\times$ embedder results grid.

Table 20: Full clustering results: all strategy $\times$ embedder combinations on CVEFixes ($n \approx 94$ per strategy).

Strategy	Embedder	ARI	Homog.	NMI	n
token_guided	GIN	0.015	0.064	0.071	94
token_guided	GIN-enriched	-0.013	0.037	0.046	94
token_guided	Combined	-0.001	0.057	0.057	94
token_guided	Combined-enriched	-0.001	0.057	0.057	94
token_guided	codebert_pattern	0.068	0.154	0.154	94
token_guided	codebert_flow	0.011	0.076	0.077	94
token_guided	codebert_flow_pattern	0.050	0.115	0.116	94
diff_based	GIN	0.029	0.108	0.111	92
diff_based	GIN-enriched	0.038	0.090	0.093	92
diff_based	Combined	0.026	0.091	0.096	92
diff_based	Combined-enriched	0.035	0.101	0.109	92
diff_based	codebert_pattern	0.017	0.099	0.100	92
diff_based	codebert_flow	0.018	0.094	0.094	92
diff_based	codebert_flow_pattern	-0.002	0.074	0.074	92
random	GIN	-0.011	0.073	0.080	70
random	GIN-enriched	-0.018	0.075	0.080	70
random	Combined	-0.003	0.087	0.091	70
random	Combined-enriched	-0.003	0.087	0.091	70
random	codebert_pattern	0.019	0.113	0.125	70
random	codebert_flow	0.024	0.114	0.124	70
random	codebert_flow_pattern	0.019	0.107	0.118	70

Conclusions

- 1. Unsupervised CWE family clustering does not work.** The best configuration (token_guided + codebert_pattern) achieves $ARI = 0.068$ and $MAP = 0.257$ —barely above random chance (≈ 0.20). No embedder, slicing strategy, or code ordering produces meaningful family-level clusters without supervised training.

2. **Vulnerability type is not the dominant signal.** The embedding space is dominated by function-specific and project-specific variation (code style, naming conventions, function complexity) that overwhelms the subtle structural differences between CWE families. Two null-pointer-dereference fixes in different subsystems are more different from each other than from a use-after-free fix in the same subsystem.
3. **Graph topology alone is uninformative for CWE discrimination.** All GIN/Combined variants (including enriched) produce $\text{ARI} \leq 0.038$. The topological “shape” of a vulnerability slice does not encode which *type* of vulnerability it is. This contrasts sharply with the AutoPatch results where structural embedders achieve 89% hit@1—but that task is same-CVE retrieval (matching identical structure), not cross-CVE family discrimination.
4. **Token-guided slicing helps only with CodeBERT.** The discriminative tokens (free, lock, null, etc.) provide useful signal only when processed by a language model that understands their semantics. For structural embedders, the token-guided slice is no better than random.
5. **Graph-flow ordering is counterproductive for CodeBERT.** Reordering code by BFS over flow edges destroys the syntactic locality that CodeBERT expects from pre-training. The model was trained on source code in line order; presenting scrambled statement sequences degrades representation quality.
6. **Race conditions are the most retrievable family.** $\text{P@1} = 0.550$ and $\text{MRR} = 0.732$ for `race_condition`—likely because lock/unlock API calls (`mutex_lock`, `spin_lock`) create distinctive textual patterns recognisable by CodeBERT. This suggests that vocabulary-based retrieval (keyword matching) may outperform embedding-based retrieval for CWE discrimination.
7. **Implications for the RAG system.** Cross-CWE retrieval requires either: (a) supervised contrastive learning with CWE labels (as demonstrated effective for same-CVE retrieval in Section 8.7), or (b) a fundamentally different approach such as specification-level matching (comparing vulnerability descriptions or fix-lists rather than raw code embeddings). The current unsupervised embedding pipeline is effective for same-CVE retrieval (89% hit@1) but cannot generalise to family-level discrimination.

References

- [1] K. Beyer, J. Goldstein, R. Ramakrishnan, and U. Shaft, “When is ‘nearest neighbor’ meaningful?” in *Proc. 7th International Conference on Database Theory (ICDT)*, pp. 217–235, Springer, 1999.
- [2] I. T. Jolliffe and J. Cadima, “Principal component analysis: a review and recent developments,” *Philosophical Transactions of the Royal Society A*, vol. 374, no. 2065, p. 20150202, 2016.
- [3] V. Raunak, V. Gupta, and F. Metze, “Effective dimensionality reduction for word embeddings,” in *Proc. 4th Workshop on Representation Learning for NLP (RepL4NLP)*, pp. 235–243, ACL, 2019.
- [4] J. Park, J. Lee, and S. Ryu, “AutoPatch: Automated Generation of Hotpatches for Real-Time Embedded Devices,” *arXiv preprint arXiv:2404.xxxxx*, 2024.
- [5] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “BLEU: a method for automatic evaluation of machine translation,” in *Proc. 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 311–318, 2002.
- [6] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, and S. Ma, “CodeBLEU: a method for automatic evaluation of code synthesis,” *arXiv preprint arXiv:2009.10297*, 2020.
- [7] C.S. Xia, Y. Wei, and L. Zhang, “Automated program repair in the era of large pre-trained language models,” in *Proc. 45th International Conference on Software Engineering (ICSE)*, pp. 1482–1494, IEEE, 2023.
- [8] Y. Li, S. Wang, and T.N. Nguyen, “GraphFVD: Fine-Grained Vulnerability Detection via Graph-Based Code Representation,” *IEEE Transactions on Software Engineering*, 2024.